

"Strings - Continuation"

* String Methods

Constructors

- new String()
- new String(String)
- new String(char[])
- new String(byte[])
- new String(StringBuilder)
- new String(StringBuffer)

* String methods

length/ Emptiness

- length()
- isEmpty()
- isBlank()

character Access

- charAt(int)
- toCharArray()

Comparison

- equals()
- equalsIgnoreCase()
- compareTo()

Searching

- contains()
- indexOf()
- lastIndexOf()
- startsWith()
- endsWith()

Conversion

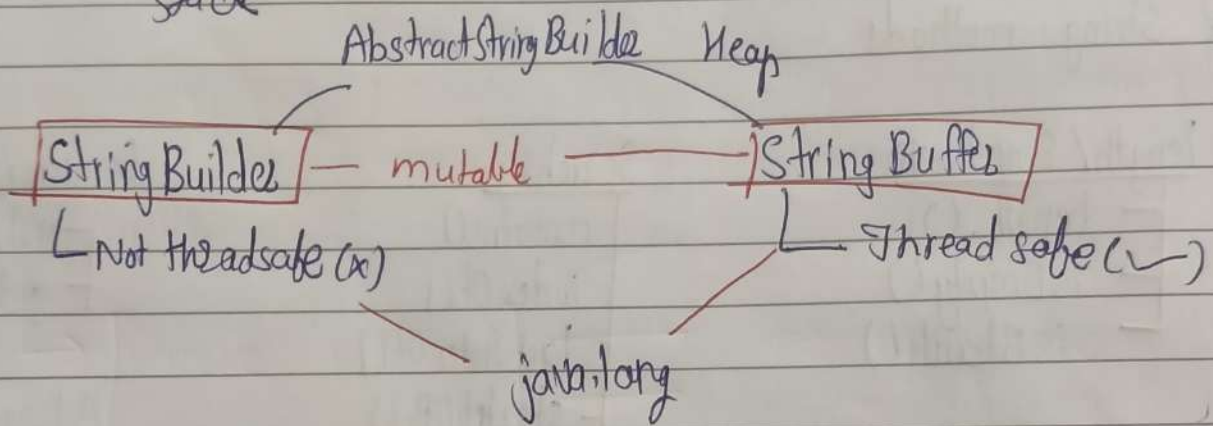
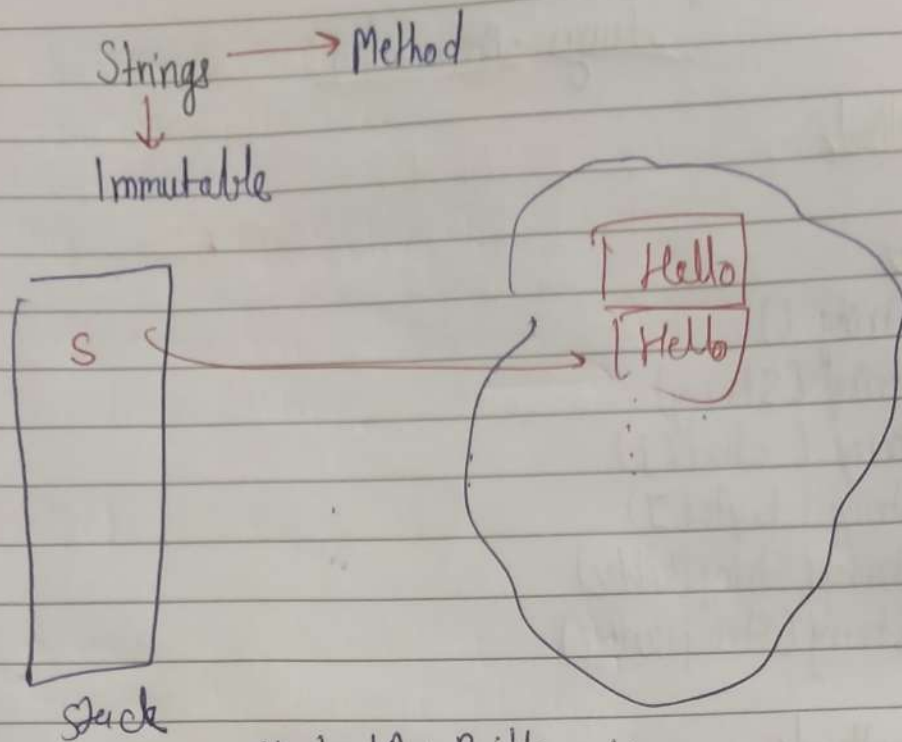
- valueOf()
- getBytes()

Advance

- intern()
- format()

Extraction / Transformation

- substring()
- toUpperCase()
- toLowerCase()
- trim()
- strip()
- repeat()
- replace()
- replaceAll()
- split()
- join()



```

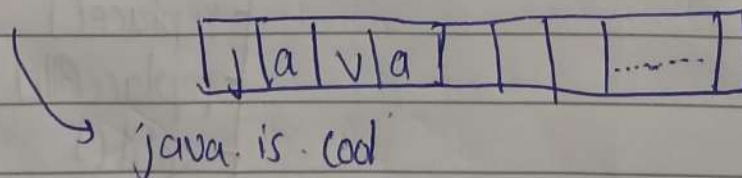
class String Builder extends AbstractString Builder {
    byte [] values; // code
    int count;
    }
    
```

initial capacity = 16

count = 4

```

String Builder sb = new String Builder ("java");
sb.append ("is cool"); // count = 12
    
```

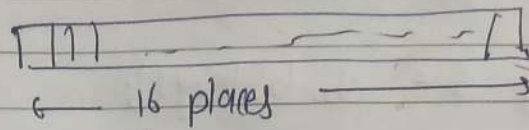


StringBuilder sb = new StringBuilder();

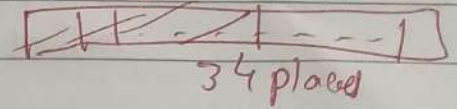
→ java

sb.append("is cool");

sb.append(" ");



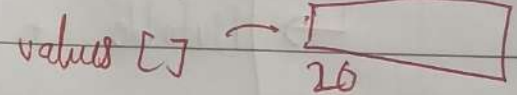
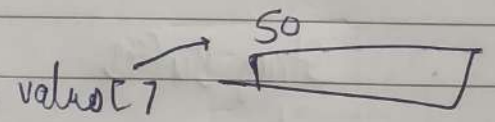
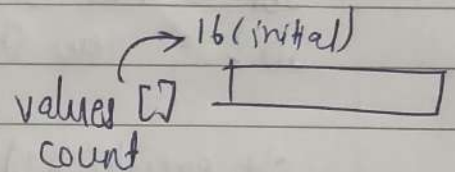
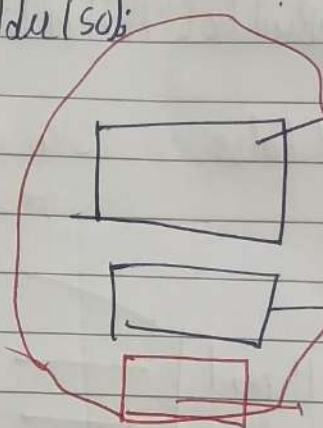
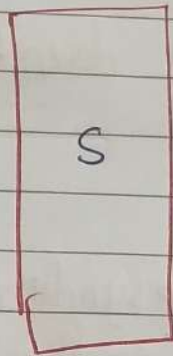
(initial * 2) + 2



String Builder

s = new StringBuilder();

~~new~~ new StringBuilder("so");



→ new StringBuilder("java");

↳ 16 + 4 = 20

String Builder / StringBuffer methods

→ append()

→ ensureCapacity()

→ insert()

→ trimToSize()

→ delete()

→ replace()

→ reverse()

→ charAt()

→ setCharAt()

→ length()

→ capacity()

String Builder

String Buffer

methods

```

sb = new StringBuilder();
sb.append("Hello");

String s = sb.toString();

```

transformation / comparison
methods of String $\rightarrow$ doesn't ~~at~~ come in
StringBuilder / StringBuffer

equals()

```

sb1 = new StringBuilder("Aditya");
sb2 = new StringBuilder("Aditya");

```

sb1.equals(sb2) // false StringBuilder doesn't override equals
method

String Buffer

$\rightarrow$ [Mutable + Thread safety] $\rightarrow$ ~~also~~ synchronize
methods for thread
safety

Thread 1

Thread 2

StringBuilder sb = new StringBuilder("Hello");

Thread 1: sb.append("A");

Thread 2: sb.append("B");

$\rightarrow$ HelloA

$\rightarrow$ HelloAB

Rate condition

| HelloB

| Hello BA

String Builder

$\gg$
faster

String Buffer

usage: $\downarrow$
90%

$\downarrow$
10% $\rightarrow$ overhead